SPACE DASH

Navigate | Shoot | Survive

SE4031 - Games Development | Assignment 01 (Individual)

Student Name:
Perera M N

IT Number:
IT22360014

1. Game Title

Space Dash

Why "Space Dash"?

The title Space Dash was chosen deliberately to reflect the core identity, setting, and gameplay feel of the game across every dimension:

- Space - The game is set in the depths of outer space. The player pilots an astronaut through a galaxy filled with floating asteroid rocks, incoming enemy bombs, starfield nebula backgrounds, and the vast emptiness between obstacles. The word "Space" immediately communicates the sci-fi setting and the sense of infinite danger in all directions.
- Dash - The word "Dash" captures the frantic, high-speed nature of survival gameplay. The player is constantly in motion, reacting at speed to gaps, rocks, enemies and health packs. As the game progresses the world scrolls faster, making every second a true dash for survival. There is no stopping, no slowing down — only forward momentum.
- Together “Space Dash" is punchy, memorable, and immediately communicates what kind of game this is. Two words, one clear promise: a fast-paced space survival experience. It echoes the spirit of classic arcade titles like Flappy Bird, Geometry Dash and Space Invaders while establishing its own identity.

Thematic Connection

Every element of the game reinforces the title:

- The astronaut player character is literally dashing through space.
- The Starfield Parallax background creates the sensation of high-speed space travel.
- The magical glowing rocks (Magical Rock series) and bomb enemies add cosmic danger.
- The speed increase over time makes each run feel like a desperate final dash.
- The health gradient bar fading from green to red mirrors the tension of surviving in a hostile space environment.

2. Student Details

Student Name	Perera M N
IT Number	IT22360014
Module	SE4031 - Games Development
Assignment	Assignment 01 (Individual)
Game Title	Space Dash
Game Engine	Unity 2022.3.20f1 (DX11)
Language	C#
Platform	Windows PC (exe build)
Submission Year	2026

3. Summary of the Game Concept

Space Dash is a 2D endless survival game set in deep space, built using Unity 2022. The player controls an astronaut character navigating a gauntlet of scrolling green tunnel walls, floating magical asteroid rocks, and incoming enemies bomb. The objective is to survive as long as possible, pass through gaps in obstacles to score points, shoot enemies with laser projectiles, and collect health packs to stay alive as the world gets faster and more dangerous.

The game takes direct inspiration from classic arcade titles such as Flappy Bird and Geometry Dash, expanding the one-button formula with a full combat system, health resource management, dynamic difficulty scaling, and a rich space visual theme built from professionally designed 2D assets.

Core Gameplay Loop

- Astronaut player flies through gaps in scrolling green block walls → +1 score per gap
- Magical glowing asteroid rocks float in randomly from the right → avoid or take damage
- Bomb enemies fly in from the right → shoot with laser for +3 score OR avoid
- Green health pack icons appear → fly through to restore 1 health point
- Left click fires a laser projectile → destroys bomb enemies on contact
- Speed increases over time → each run gets faster and harder
- Take 3 hits total → health reaches 0 → Game Over screen with final score

Unique Features

- Dual obstacle system — wall gaps AND random floating asteroid rocks create layered challenge
- Astronaut character with directional animations (gun_up / gun_side / gun_down)
- 5 different magical rock prefabs spawning in random positions and rotations
- Screen shake + gradient health bar as creative enhancements
- Vibrant 4K starfield galaxy background with parallax depth

4. Description of Core Features

4.1 Health System (Requirement 1)

The player starts with 3 health points displayed as a UI Slider bar in the top left. The health bar fill color smoothly interpolates from green (full) through yellow (half) to red (critical) every frame using Color.Lerp() in GameManager.UpdateHealthUI(). Each collision with a wall, asteroid rock, or bomb enemy removes 1 health point. An invincibility coroutine in PlayerController.cs (1.5 seconds) prevents multiple rapid hits registering at once and causes the player to flash visually. The player physically passes through walls after collision rather than bouncing. When health reaches 0, Die () is called: movement freezes, gravity disables, and TriggerGameOver() shows the Game Over panel. Health resets to maxHealth (3) on restart via SceneManager.LoadScene().

4.2 score System (Requirement 2)

Score is displayed as a TMP_Text UI element in the top-right corner. A ScoreTrigger child object with a Box Collider 2D (Is Trigger enabled) sits inside the gap of each WallPair prefab. When the Player-tagged object passes through, OnTriggerEnter2D fires in ScoreTrigger.cs and calls GameManager.AddScore(1). Destroying a bomb enemy with a laser projectile awards +3 score. The score text updates immediately via UpdateScoreUI(). Score resets to 0 on restart.

4.3 Health Packs (Requirement 3)

Health packs use the Deco_03 prefab from the Comic Battle Royale asset pack as their visual. Each health pack has a Box Collider 2D (Is Trigger), the HealthPack tag, and the HealthPack.cs script. They are spawned by WallSpawner at a 35% chance alongside each wall pair. On trigger collision with the player, HealPlayer(1) is called (capped at maxHealth = 3) and the pack is destroyed immediately. If the player misses it, destroy (gameObject, 7f) in Start () auto-destroys it after 7 seconds. They also self-destroy when moving off the left edge of the screen.

4.4 Projectile Attacks (Requirement 4)

Left mouse click instantiates the Projectile prefab at the player's current world position. The projectile is a laser bolt sprite with Rigidbody2D (gravity 0), Circle Collider 2D (Is Trigger), and a rightward velocity of Vector2.right * 10f applied in Start (). It auto-destroys after 2 seconds via Destroy (gameObject, lifetime). On trigger collision with a Bomb enemy: GameManager.AddScore(3) is called, the enemy is destroyed, and the projectile is destroyed. On collision with anything else (walls, rocks): the projectile is destroyed immediately.

4.5 Enemies (Requirement 5)

Enemies use the bomb prefab from the Comic Battle Royale asset pack as their visual, making them clearly distinguishable from health packs (Deco_03) by both shape and color. Each enemy has a Rigidbody2D (Kinematic, gravity 0), Circle Collider 2D (Is Trigger), the Enemy tag, and Enemy.cs. They are spawned by WallSpawner at a 45% chance. Projectile collision: +3 score, both destroyed. Player collision: player loses 1 health (handled in PlayerController.OnTriggerEnter2D), enemy destroyed. Auto-destroy after 7 seconds or when off screen left.

4.6 Speed Increase Over Time (Requirement 6)

currentSpeed is recalculated every frame in GameManager.Update() using: currentSpeed = Mathf.Min(baseSpeed + score * 0.1f + Time.timeSinceLevelLoad * 0.03f, maxSpeed). The base speed starts at 3f and is capped at 8f. Speed scales with both the player's score and elapsed time, ensuring the game accelerates whether the player is skilled or struggling. WallMover reads GameManager.Instance.currentSpeed every frame so all active wall pairs, health packs, enemies, and asteroid rocks update their movement speed in real time. WallSpawner also reduces its spawn interval as score increases (min 1.2s), making the game progressively more intense without becoming unplayable.

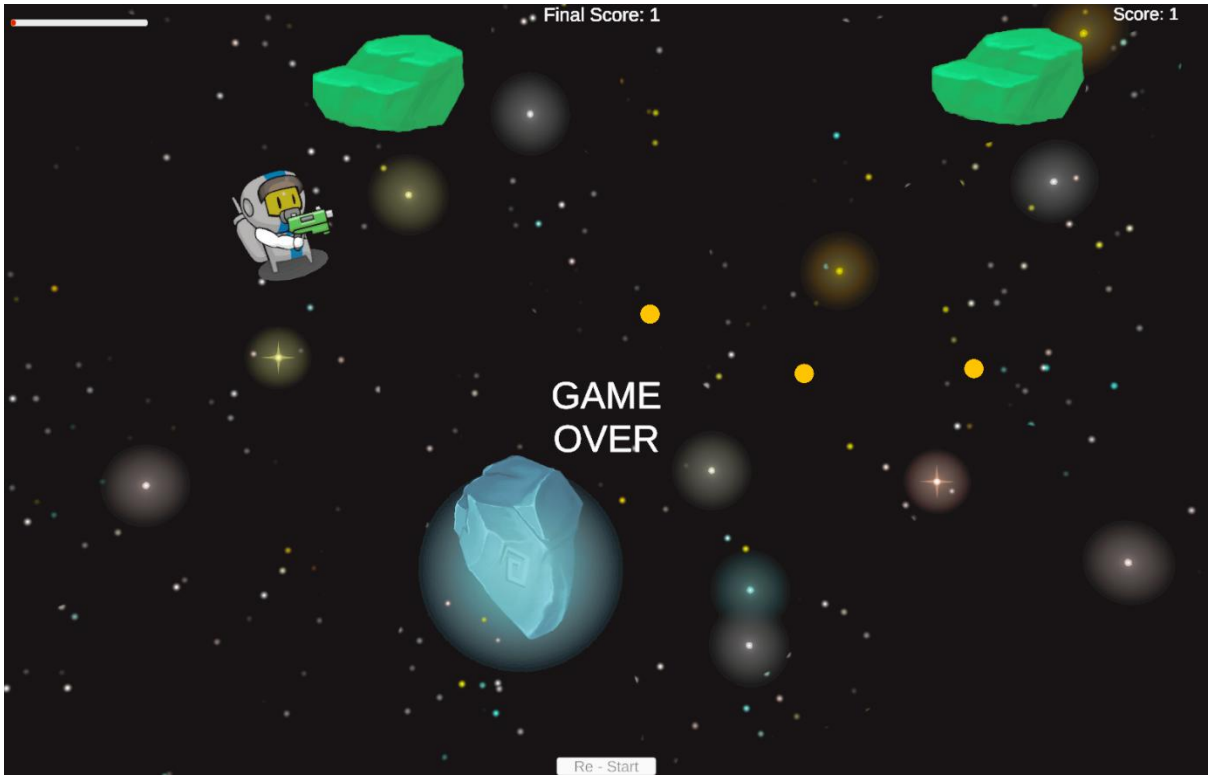
4.7 Creative Features (Requirement 7)

Two creative features were implemented:

(1) Gradient Health Bar: The health bar fill Image color updates every frame using a continuous Color.Lerp() calculation based on the exact currentHealth/maxHealth ratio. Above 50% health it interpolates from yellow to green; below 50% it interpolates from red to yellow. This produces a smooth, always-accurate color gradient rather than jumping between fixed threshold colors. It provides instant visual health awareness without reading numbers.

(2) Screen Shake on Damage: The ScreenShake.cs script attached to the Main Camera runs a coroutine whenever TakeDamage() is called. The camera's localPosition is randomly offset within a magnitude of 0.12f over 0.25 seconds, then smoothly returned to its original position. This creates a strong physical impact sensation for every hit, reinforcing the danger of collision with walls, rocks, and enemies.

5. Screenshots of Gameplay



6. Control Guide

Input	Action
Space Bar	Flap upward — primary flight control
Right Mouse Button	Flap upward — alternative flight control
Left Mouse Button	Fire laser projectile from player position
Restart Button (UI)	Restart game after Game Over — resets health, score, speed

Space Dash uses an intentionally minimal two-action control scheme. The challenge comes entirely from spatial awareness, timing, and resource management rather than complex inputs, keeping the game accessible to any player while remaining demanding at higher speeds.

Gameplay Tips

- Use short, controlled taps on Space rather than holding - the astronaut responds to each press
- Stay near vertical center of the screen - gives maximum reaction time for rocks and bombs
- Shoot bombs proactively before they reach you - +3 score and zero damage is always the best trade
- Watch the health bar color - when it turns yellow start actively hunting health packs
- Speed increases gradually - the first 30 seconds are the best time to build up score safely

7. Creative Feature Description

Feature 1 - Dynamic Gradient Health Bar

The health bar fill color is recalculated and updated on every single frame based on the player's exact current health ratio. Unlike a simple threshold approach (e.g. red below 33%), the gradient is continuous and always perfectly reflects the player's exact health state.

```
// In GameManager.UpdateHealthUI() — runs every frame
float t = (float)currentHealth / maxHealth;
if (t > 0.5f)
    color = Color.Lerp(Color.yellow, Color.green, (t-0.5f)*2f);
else
    color = Color.Lerp(Color.red, Color.yellow, t*2f);
healthBarFill.color = color;
```

Result: at 3/3 health the bar is bright green. At 2/3 it is yellow-green. At 1/3 it is bright red. The transition is perfectly smooth and continuous.

Feature 2 - Screen Shake on Damage

Every time the player takes damage — from a wall, an asteroid rock, or a bomb enemy — the Main Camera runs a screen shake coroutine via ScreenShake.cs. The camera's localPosition is randomly offset within a magnitude of 0.12f for 0.25 seconds (approximately 12-15 random positions), then returned to its exact original position. The effect is triggered by GameManager.TakeDamage() calling ScreenShake.Instance.Shake(). This creates immediate, visceral physical feedback for every hit and makes the game feel much more impactful than a simple health number decrement.

8. Assets Used - Characters, Environment & Objects

8.1 Player Character - Astronaut

Asset Pack	Comic Battle Royale — 2D Character Astronaut
Source	Unity Asset Store (imported via Package Manager)
Variant Used	Variant A
Prefab Location	COPY SPRIGHT → Comic Battle Royale → 2D Character - Astronaut → Variant A → Prefabs → Character
Animations Used	gun_side (default flight), gun_up (ascending), gun_down (descending)
Collider	Capsule Collider 2D fitted to astronaut body
Script	PlayerController.cs — handles flap, shoot, collision, invincibility, flash
Tag	Player

8.2 Enemy - Bomb

Asset Pack	Comic Battle Royale — 2D Character Astronaut
Source	Unity Asset Store
Prefab Used	Bomb
Prefab Location	COPY SPRIGHT → Comic Battle Royale → Variant A → Prefabs → Objects → Bomb
Visual Style	Round black bomb with fuse — clearly distinct from health pack
Collider	Circle Collider 2D (Is Trigger) on parent Enemy object
Script	Enemy.cs — handles movement, lifetime, projectile collision
Tag	Enemy
Behaviour	Flies in from right, deals 1 damage on player contact, destroyed by laser (+3 score)

8.3 Health Pack - Deco_03

Asset Pack	Comic Battle Royale — 2D Character Astronaut
Source	Unity Asset Store
Prefab Used	Deco_03
Prefab Location	COPY SPRIGHT → Comic Battle Royale → Variant A → Prefabs → Objects → Deco_03
Visual Style	Decorative green object — clearly distinct from bomb enemy by color and shape
Collider	Box Collider 2D (Is Trigger) on parent HealthPack object
Script	HealthPack.cs — handles movement, healing, lifetime auto-destroy
Tag	HealthPack
Behaviour	Restores 1 health on collection (capped at 3), auto-destroys after 7 seconds

8.4 Obstacle Rocks - 5 Magical Rock Prefabs

Asset Pack	2D Handpainted Magical Rocks and Stones
Source	Unity Asset Store (imported via Package Manager)
Prefab Location	2D_Handpainted_Magical_Rocks_and_Stones → Prefabs → Blue
Prefabs Used	Magical_Rock_4, Magical_Rock_7, Magical_Rock_17, Magical_Rock_18, Magical_Rock_20
Visual Style	Handpainted blue/purple glowing magical rocks — space asteroid feel
Collider	Box Collider 2D (Is Trigger) on parent Obstacle object
Script	Rock.cs — random rotation speed, movement, lifetime, collision handling
Tag	Wall
Behaviour	Spawned randomly by RockSpawner — max 3 on screen, always passable gap, slow tumbling rotation

8.5 Wall Obstacles - Block_01

Asset Pack	Comic Battle Royale — 2D Character Astronaut
Source	Unity Asset Store
Prefab Used	Block_01
Visual Style	Solid comic-style block — used as top and bottom tunnel walls
Collider	Box Collider 2D on each wall segment
Tag	Wall
Behaviour	Spawned in pairs with a random gap position by WallSpawner every 1.2–2.5 seconds

8.6 Environment - Space Background

Asset Pack	Vibrant 4K Starfield Skybox
Source	Unity Asset Store (ParallelCascades)
Textures Used	Simple Starfield with Flaring Stars, Vibrant Space Skybox 1
Implementation	Textures converted to Sprite (2D and UI), placed as background GameObjects with Order in Layer -10 and -9
Effect	Deep space starfield with galaxy nebula — creates immersive space atmosphere

8.7 Projectile

Visual	Orange circle sprite (Unity built-in 2D primitive)
Collider	Circle Collider 2D (Is Trigger)
Physics	Rigidbody2D, gravity 0, velocity Vector2.right * 10f
Script	Projectile.cs
Tag	Projectile
Behaviour	Fires right on left-click, destroys bomb enemy (+3 score), auto-destroys after 2 seconds

9. Credits

Game Engine & Development Tools

- Unity 2022.3.20f1 (DX11) Game development engine
- C# - Primary programming language
- Visual Studio / JetBrains Rider — Code editor
- Unity TextMeshPro - UI text rendering
- GitHub - Version control and assignment submission
- Unity Package Manager - Asset import and management

External Asset Packs Used

- Comic Battle Royale - 2D Character Astronaut - Unity Asset Store - Player character, Bomb enemy, Deco_03 health pack, Block_01 walls
- 2D Hand painted Magical Rocks and Stones - Unity Asset Store - 5 asteroid rock obstacle prefabs
- Vibrant 4K Starfield Skybox (ParallelCascades) - Unity Asset Store - Space background textures

References

- Unity Manual - https://docs.unity3d.com/Manual/Quickstart2DCreate.html
- Unity Scripting API - https://docs.unity3d.com/ScriptReference/
- Unity TextMeshPro Documentation - https://docs.unity3d.com/Packages/com.unity.textmeshpro@3.0/manual/

10. Demo Video & Unity Project Links

Video Link	IT22360014_Demo Video
Unity Project	IT22360014_UnityProject